RDBMS & Database Programming With JDBC

Answer Key — Food Delivery Console App Assessment

SECTION A — Conceptual Understanding

Q1. Compare HashMap and ArrayList for active orders.

For active orders keyed by order ID, **HashMap<String, Order>** is the more suitable structure. A HashMap gives expected O(1) lookup, insertion, and removal by key, assuming a well-distributed hash function. With an ArrayList, locating an order by ID normally requires a linear scan, so lookup and removal by ID are O(n); append is amortized O(1). Therefore, replacing the HashMap would make the frequent ID-based operations less direct and potentially slower as the number of active orders grows. ArrayList is more appropriate when the platform mainly needs ordered/sequential access, such as displaying a customer's cart or a pre-sorted list of restaurant menu items.

Logic:

- HashMap is keyed by order ID, so get(id), put(id, order), and remove(id) are expected O(1).
- ArrayList preserves sequence and supports indexed access, but finding an order by ID is O(n).
- Use ArrayList where iteration/order matters more than key-based lookup.
- The HashMap O(1) figures are expected/average-case; pathological hash collisions can change the practical cost.

Q2. Explain a custom exception hierarchy for payment processing.

A custom exception hierarchy models related business failures as specific exception types instead of catching one generic **Exception** everywhere. This makes the calling code easier to read because it can handle insufficient funds, a gateway timeout, and an unavailable item differently. A common checked base class such as **PaymentException** can extend **Exception**, with specific subclasses representing each business failure.

```
class PaymentException extends Exception {    public PaymentException(String message) {        super(message);    }}class InsufficientBalanceE
```

Logic:

- PaymentException groups payment-related checked failures under one meaningful type.
- InsufficientBalanceException represents a wallet/business-rule failure.
- PaymentGatewayTimeoutException represents an external service timeout.
- ItemUnavailableException represents a restaurant/menu availability failure.
- Specific catch blocks can show the correct user message or retry only the failures that are retryable.

Q3. Explain SQL Injection and how PreparedStatement prevents it.

String concatenation lets attacker-controlled text become part of the SQL syntax. For example, if a login query is built as **SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"**, an input such as **' OR '1'='1** can alter the WHERE expression so that it evaluates as true. A **PreparedStatement** separates the SQL template from parameter values: the placeholders are part of the statement structure, while setString() supplies data. The parameter is therefore not reinterpreted as executable SQL syntax.

```
// VulnerableString sql = "SELECT * FROM users WHERE username='" + username          + "' AND password='" + password + "'";Statement st = con
```

Logic:

- Statement commonly receives a complete SQL string after application concatenation.
- PreparedStatement uses placeholders and binds parameter values separately.
- A value such as ' OR '1'='1 is treated as a String value, not as new SQL operators.
- PreparedStatement should be used for user-supplied values rather than concatenating them into SQL.

Q4. Explain JDBC transaction management for atomic order placement.

With auto-commit enabled, each JDBC update can be committed independently. For example, the wallet debit may succeed and the order row may be inserted, but the restaurant counter update can then fail because the restaurant ID does not exist. The database would contain a partially completed order. The solution is to use one Connection, disable auto-commit, execute all three statements, and commit only after all succeed. If any statement fails, rollback() must undo the earlier changes.

```
try (Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD)) {    conn.setAutoCommit(false);    try {        try (Prepare
```

Logic:

- setAutoCommit(false) makes the three statements part of one transaction.
- commit() makes all successful changes permanent together.
- rollback() restores the state from before the transaction if a later statement fails.
- All three operations must use the same Connection for the transaction boundary to apply.

Q5. Explain safe ResultSet column access and SQL NULL handling.

Reading columns by numeric position can become fragile when a SELECT list changes order or when a new column is inserted. It also makes code less readable because rs.getString(2) does not explain which field is being read. Column labels are preferred, for example rs.getString("restaurant_name") and rs.getBigDecimal("total_amount"). JDBC getter methods return Java null for object/reference types when the SQL value is NULL; primitive-style getters such as getInt() can return a default value such as 0, so **wasNull()** should be checked immediately after the getter when NULL has semantic meaning.

```
while (rs.next()) {    String restaurant = rs.getString("restaurant_name");    BigDecimal amount = rs.getBigDecimal("total_amount");    Date o
```

Logic:

- Column labels make code more readable and less sensitive to SELECT-column reordering.
- getString()/getBigDecimal()/getDate() return null for SQL NULL where the Java return type is a reference type.
- Primitive getters cannot return null, so JDBC provides wasNull() to distinguish SQL NULL from a real zero/false value.
- Call wasNull() immediately after the getter whose SQL NULL status you want to inspect.

Q6. Explain JDBC Batch Processing for 500 inserts.

Calling executeUpdate() inside a loop can require a database interaction for every row, so 500 inserts may involve roughly 500 statement executions/round trips, depending on the driver and its batching behaviour. Batch processing groups many parameter sets and sends them together, reducing client/server communication overhead. The normal sequence is: prepare the statement, set parameters, call addBatch() for each row, call executeBatch(), and commit the transaction. If the driver reports a BatchUpdateException, inspect its update counts and decide whether the transaction should be rolled back.

```
String sql =    "INSERT INTO report_logs (order_id, report_date) VALUES (?, ?)";try (Connection conn = DriverManager.getConnection(DB_URL, DB_
```

Logic:

- addBatch() queues one parameter set for the prepared statement.
- executeBatch() sends the queued batch to the database and returns update counts.
- commit() makes the batch changes permanent when auto-commit is disabled.
- BatchUpdateException can contain update counts showing which entries were processed before the reported failure.
- Driver behaviour differs, so exact network round trips and partial-success semantics depend on the JDBC driver/database.

SECTION B — Practical Tasks

QB1. MenuItem Class with Encapsulation

The class keeps all four fields private, initializes the default constructor as required, and exposes only the setters permitted by the assessment. itemId and name therefore remain read-only after construction.

```
public class MenuItem {    private int itemId;    private String name;    private double price;    private boolean isAvailable;    public Menu
```

Logic:

- private fields enforce encapsulation.
- The default constructor sets price to 0.0 and availability to true.
- Only price and availability have setters, matching the read-only requirement for itemId and name.
- displayDetails() adds [UNAVAILABLE] immediately after the item name when required.

QB2. CartManager with Collections and Custom Exceptions

CartManager stores menu items in an ArrayList and enforces the two required business rules: unavailable items cannot be added, and removing a missing ID throws NoSuchElementException. The custom ItemUnavailableException is checked because it extends Exception.

```
import java.util.ArrayList;import java.util.NoSuchElementException;class ItemUnavailableException extends Exception {    public ItemUnavailabl
```

Logic:

- ArrayList provides dynamic storage and simple iteration for the cart.
- ItemUnavailableException communicates a specific business-rule failure.
- removeItem() scans the list by item ID and throws NoSuchElementException when no match exists.
- The final line prints the calculated grand total; an empty cart prints exactly Cart is empty.

QB3. JDBC Order Logger with PreparedStatement

The logger uses DriverManager.getConnection(), PreparedStatement for both INSERT and SELECT, generated keys for the new order ID, and try-with-resources for JDBC cleanup. The returned history is an ArrayList using the required format.

```
import java.sql.*;import java.util.ArrayList;public class OrderLogger {    private static final String DB_URL =        "jdbc:mysql://localhost
```

Logic:

- PreparedStatement keeps customer input out of the SQL syntax.
- RETURN_GENERATED_KEYS retrieves the database-generated order ID.
- try-with-resources closes Connection, PreparedStatement, and ResultSet automatically.
- The SELECT is parameterised by customerId and returns the requested formatted strings.

QB4. Atomic Order Placement with Transaction Control

All three database operations run on one Connection with auto-commit disabled. If the restaurant update fails, the catch block rolls back the wallet deduction and order insertion as well, so the database does not retain a partial order.

```
import java.sql.*;public class OrderService {    private static final String DB_URL =        "jdbc:mysql://localhost:3306/food_delivery";    p
```

Logic:

- The same Connection owns the complete transaction.
- The third UPDATE affects zero rows when the restaurant ID does not exist, so it is treated as a failure.
- rollback() undoes both the earlier wallet deduction and order INSERT.
- The main() call uses a non-existent restaurant ID to demonstrate the rollback path requested by the assessment.

SECTION C — Mini Project

QC1. Food Delivery Console App — Design and Database Setup

The application should combine MenuItem encapsulation, an ArrayList-based cart, JDBC PreparedStatement queries, and transaction control. A minimal MySQL schema can support the required menu, customer wallet, restaurant counter, and order-history features.

```
CREATE TABLE menu_items (    item_id INT PRIMARY KEY AUTO_INCREMENT,    name VARCHAR(100) NOT NULL,    price DECIMAL(10,2) NOT NULL,    is_ava
```

Logic:

- menu_items supplies the startup menu and availability status.
- customers supplies wallet_balance for the transactional order placement.
- restaurants supplies the pending_orders counter.
- orders stores order history including amount and date.

QC2. Load the Menu with JDBC and ArrayList

At startup, use a PreparedStatement SELECT and convert each ResultSet row into a MenuItem object stored in an ArrayList. The menu can then be displayed without repeatedly querying the database.

```
public static ArrayList<MenuItem> loadMenu(Connection conn)        throws SQLException {    ArrayList<MenuItem> menu = new ArrayList<>();    S
```

Logic:

- PreparedStatement is used even though this SELECT has no input parameters, keeping the JDBC pattern consistent.
- Each row becomes one MenuItem object.
- The ArrayList gives the console program an in-memory representation of the menu.

QC3. Atomic Place-Order Operation

The order flow must deduct the wallet amount and insert the order inside one transaction. If either operation fails, rollback() restores the previous wallet and order state. A clear error message should be shown instead of allowing the console program to crash.

```
public static void placeOrder(Connection conn,                                String customerId,                                double amount)
```

Logic:

- The wallet UPDATE checks that enough balance exists before the debit is committed.
- The order INSERT is part of the same transaction.
- Any SQLException causes rollback(), so the two changes succeed or fail together.
- The finally block restores auto-commit for later independent operations on the Connection.

QC4. Order History and Menu-Driven Console Loop

The console program should repeatedly present the four required options, validate input, and call the relevant operation. Order history is parameterised by customer ID and displayed as a formatted table.

```
public static void showOrderHistory(Connection conn,                                        String customerId)        throws SQLException {    S
```

Logic:

- The loop continues until option 4 is selected.
- PreparedStatement prevents customerId from becoming SQL syntax.
- Empty order history prints No orders found.
- Non-numeric input and negative amounts should be validated before database operations.

SECTION D — AI-Augmented Learning

QD1. Step 1 — Exact AI Prompt

A suitable prompt should reproduce every requirement in the assessment so the generated program can be tested against it.

```
Write a complete Java JDBC console program that connects to a MySQLdatabase and reads all rows from a table called food_orders.Requirements:1.
```

Logic:

- The prompt explicitly includes the table, grouping, revenue, count, formatting, empty-table behaviour, and resource handling requirements.
- The resulting code should still be tested manually because AI-generated code can contain design or resource-management issues.

QD2. Step 2 — Example of the AI's Original Code

A plausible first version may read all rows and perform the grouping in Java with a HashMap. It can work, but it is less efficient than letting the database perform the aggregation and may also mishandle NULL restaurant names or resource cleanup.

```
Connection conn = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);Statement st = conn.createStatement();ResultSet rs = st.executeQue
```

Logic:

- The aggregation is performed in Java instead of SQL, so more rows are transferred to the application.
- Manual close() calls are vulnerable to being skipped if an exception occurs before the close statements.
- A SQL NULL restaurant_name becomes a null HashMap key, which may not match the desired report format.
- This version is useful as a test/debug starting point, but it has clear improvement opportunities.

QD3. Step 3 — Corrected Version

The corrected version pushes grouping and aggregation into SQL, uses PreparedStatement and try-with-resources, handles an empty result set, and gives NULL restaurant names a clear label. COALESCE ensures the revenue expression is numeric.

```
import java.sql.*;public class RestaurantRevenueReport {    private static final String DB_URL =        "jdbc:mysql://localhost:3306/food_deli
```

Logic:

- GROUP BY and SUM/COUNT let MySQL perform the aggregation instead of transferring every row to Java.
- try-with-resources closes the JDBC objects automatically, including when an exception is thrown.
- A NULL restaurant name is displayed as [UNKNOWN] instead of being used blindly as a report label.
- The found flag handles the empty-table case and prints No orders found.
- COALESCE makes the revenue aggregate explicitly numeric when applicable.

QD4. Step 4 — Debugging / Improvement Note

The original AI solution manually closed JDBC resources, so an exception could leave a connection or ResultSet open. I changed it to try-with-resources so cleanup is automatic. I also moved the grouping into a SQL GROUP BY query, reducing the amount of data transferred to Java. Finally, I handled NULL restaurant names and the empty-result case explicitly so the report does not fail or print an unclear value.

Logic:

- The corrected version directly addresses the assessment's listed debugging targets.
- The final submission should include the exact prompt, the original AI code, the corrected code, and the 3–4 line change note.

Submission reminder: For Section D, submit all three required items: the exact AI prompt(s), the AI-generated/original code and corrected version, and the short note explaining what was changed and why.